Readers–Writers Synchronization Project

Student Name: Molka Derouich

Student ID: 230504599

1. Introduction

In this project, I worked on the readers–writers synchronization problem, which is a well-known problem in operating systems. The idea is simple: several processes want to access the same shared data. Readers are allowed to read the data at the same time, but writers must work alone. Without proper synchronization, this can cause data inconsistency and race conditions, so a correct solution is required.

2. Project Requirements

The program follows these rules:

When a writer is writing, no other reader or writer is allowed to access the data.

More than one reader can read the data at the same time.

A writer must wait until all readers finish reading.

Writers are not allowed to overwrite data that has not been read yet.

Each reader reads the same data only once.

3. Semaphore Implementation

To control access to shared resources, I implemented my own semaphore class using Java's wait() and notify() methods. The semaphore has acquire() and release() functions, which allow threads to safely wait and continue when resources become available.

4. Read–Write Lock Implementation

The ReadWriteLock class is responsible for managing access to the shared data. I used one semaphore as a mutex to protect the reader counter and another semaphore to give writers exclusive access. When the first reader starts reading, writers are blocked. When the last reader finishes, writers are allowed to continue. This way, readers can work in parallel while writers are still protected.

5. Shared Data and Read-Once Guarantee

To make sure that readers do not read the same data more than once, I used a version number with the shared data. Every time a writer updates the data, the version number increases. Each reader remembers the last version it read and only reads the data if a newer version exists. This also prevents writers from overwriting data that has not been read.

6. Reader and Writer Threads

Readers and writers are implemented as separate thread classes.

A reader first acquires the read lock, reads the data, and then releases the lock.

A writer acquires the write lock, updates the data, and then releases the lock.

This ensures that all threads follow the synchronization rules correctly.

7. Test Program

In the test program, I created multiple reader and writer threads and started them in different orders. This helps show that the solution works correctly even when several threads are running at the same time.

8. Sample Output

The output shows that multiple readers can read the same version of the data, writers update the data safely, and readers always get the correct and most recent value.

9. Conclusion

Overall, this project demonstrates a correct solution to the readers–writers problem using semaphores in Java. The implementation follows all the required conditions, allows safe concurrent reading, ensures exclusive writing, and produces correct results when tested.